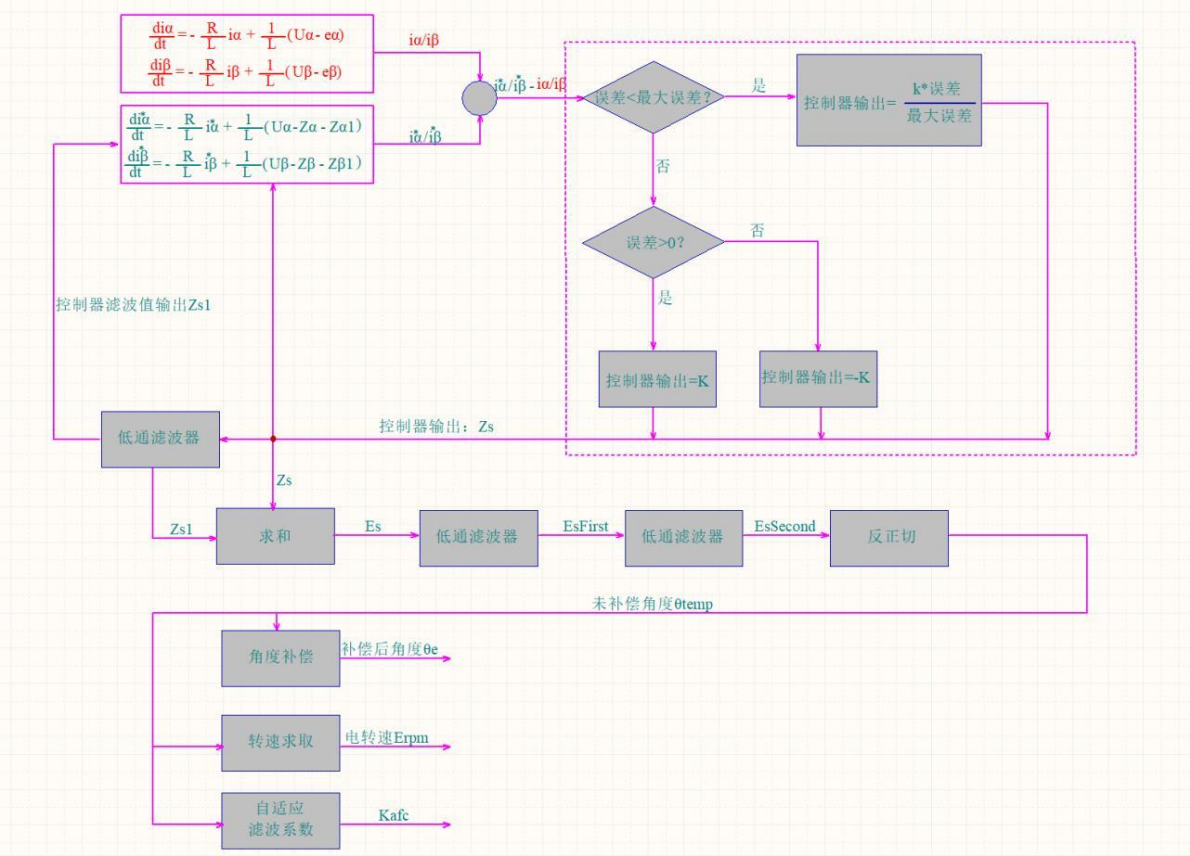
69-FOC 22

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

我们开始上课啊，截止到上次课呢，我们把电流观测器的框图分析完了。

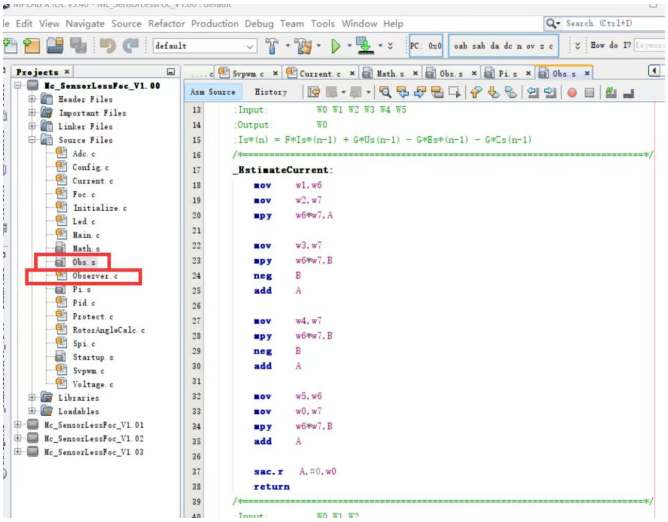


对应这幅框图呢，我们要在单片机内部代码实现。上次课呢，说过代码分了 7 个部分。

1. 电流估算方程的代码实现；
2. 电压矢量 Us 在 α β 轴上的投影 Uα，Uβ 求取；
3. 电流观测器中控制器的代码实现；
4. 数字低通滤波器的代码实现(包括自适应滤波系数的求取)；
5. 利用估算出来的反电动势，调用数字迭代算法函数，求解估算反电动势合成矢量夹角；
6. 角度补偿；
7. 估算电转速的求取。

下面呢，我们就分部分的来分析对应的代码。首先呢，我们来看下电流估算方程的代码实现。大家在电脑边上的话，可以打开一起对照来看看。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



$$F = 1 - \frac{R}{L}Ts$$

$$G = \frac{Ts}{L}$$

$$i\alpha^{*}(n) = F\ i\alpha^{*}(n-1) + G\,(U\alpha(n-1) - Z\alpha(n-1) - Z\alpha 1(n-1))$$

$$i\beta^{*}(n) = F\ i\beta^{*}(n-1) + G(U\beta(n-1) - Z\beta(n-1) - Z\beta 1(n-1))$$

电流估算方程呢，离散化之后就是这幅图这样。这样的话，我们就可以直接在单片机内部实现了。

```
typedef struct
{
    signed int qUalpha;
    signed int qUbeta;

    signed int qIalpha;
    signed int qIbeta;

    signed int qIalphaEstimate;
    signed int qIbetaEstimate;

    signed int qZalpha;
    signed int qZbeta;
    signed int qZErrMax;
    signed int qZKSlide;

    signed int qKafc;           //e(n+1) = e(n) + Kafc(z(n) - e(n))
    signed int qKafcFinal;      //es(n+1) = es(n) + KafcFinal(e(n) - es(n))

    signed int qEalpha;
    signed int qEbeta;
    signed int qEalphaFinal;
    signed int qEbetaFinal;

    signed int qTheta;
    signed int qThetaLast;
    signed int qThetaSum;
    signed int qThetaSumCnt;

    signed int qOmega;
    signed int qOmegaFilted;
    signed int qKomega;

    signed int qF;
    signed int qG;

    unsigned int Erpm;
}tObs_Type;
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

整个电流观测器部分，会用到这些变量，所以，把他们定义为这样一个结构体。

$$F = 1 - \frac{R}{L}Ts$$

$$G = \frac{Ts}{L}$$

$$i\alpha^{*}(n) = F\, i\alpha^{*}(n-1) + G\,(U\alpha(n-1) - Z\alpha(n-1) - Z\alpha1(n-1))$$

$$i\beta^{*}(n) = F\, i\beta^{*}(n-1) + G(U\beta(n-1) - Z\beta(n-1) - Z\beta1(n-1))$$

实际上大家再来看下这两个方程，我们其实可以写一个公共的函数来调用。只不过，计算 iα*或者 iβ*的话，对应的调用的参数不同而已。而这些参数呢，可以作为形参来输入，所以，两个方程的话，写一个函数就行。

```
/*=============================================================================*/
;Input:          W0 W1 W2 W3 W4 W5
;Output:         W0
;Is*(n) = F*Is*(n-1) + G*Us(n-1) - G*Es*(n-1) - G*Zs(n-1)
/*=============================================================================*/
_EstimateCurrent:
    mov     w1,w6
```

例如，大家来看这个注释。这里的 s 呢，可以表示α，也可以表示β。

```
extern signed int EstimateCurrent(signed int y,signed int g,signed int u,signed int e,signed int z,signed int f);
extern signed int QLowPassFilter(signed int y,signed int x,signed int k);
```

然后，C 文件中，调用的时候就是这样的，总共包含 6 个形参，然后，对应的返回值呢，就是要计算的估算电流。 当然了，这里为了下面的分析，我们可以先把形参对应的 W 寄存器对应出来。

```
/*=============================================================================*/
;Input:          W0 W1 W2 W3 W4 W5
;Output:         W0
;Is*(n) = F*Is(n-1) + G*Us(n-1) - G*Es(n-1) - G*Zs(n-1)
;W0 : Is(n-1) W1:G  W2:Us(n-1)  W3:Es(n-1) W4 :Zs(n-1)  W5:F
/*=============================================================================*/
_EstimateCurrent:
```

红框中就是根据形参来分配寄存器的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
;Is*(n) = F*Is(n-1) + G*Us(n-1) - G*Es(n-1) - G*Zs(n-1)
;W0 : Is(n-1) W1:G  W2:Us(n-1)  W3:Es(n-1) W4 :Zs(n-1)  W5:F
/*=============================================================================*/
_EstimateCurrent:
    mov     w1,w6
    mov     w2,w7
    mpy     w6*w7,A

    mov     w3,w7
    mpy     w6*w7,B
    neg     B
    add     A

    mov     w4,w7
    mpy     w6*w7,B
    neg     B
    add     A

    mov     w5,w6
    mov     w0,w7
    mpy     w6*w7,B
    add     A

    sac.r   A,#0,w0
    return
```

接下来呢，我们就来分析这个汇编函数。

```
_EstimateCurrent:
    mov     w1,w6 ;w6 : G
    mov     w2,w7 ;w7 : Us(n-1)
    mpy     w6*w7,A ; G*Us(n-1)
```

因为我们这里要使用 mpy 指令，它属于一条 DSP 乘法指令，它的两个乘数寄存器是有要求的。所以呢，我们要把对应的参数，传到支持的寄存器中去，这里就是 W6,W7。

问：这个 mpy 和普通的 MUL 区别主要是什么呢？
答：这个是小数乘法，mul 是整数乘法。换句话说，mpy 不需要考虑小数点问题，而 mul 需要。

问：不是已经归一化了么？
答:是的,乘数都归一化了。如果是 mpy， Q15xQ15 结果是 Q15。如果是 mul Q15xQ15 结果是 Q30， 就算取高位，还要左移一位的。而 mpy 没有这么复杂。

所以，这三条指令，就是为了计算 G*Us(n-1)，把对应的结果，存放到累加器 A 中。

那我们继续往下来看。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
;Output:        W0
;Is*(n) = F*Is(n-1) + G*Us(n-1) - G*Es(n-1) - G*Zs(n-1)
;W0 : Is(n-1) W1:G  W2:Us(n-1)  W3:Es(n-1) W4 :Zs(n-1)  W5:F
/*=============================================================================*/
_EstimateCurrent:
    mov     w1,w6 ;w6 : G
    mov     w2,w7 ;w7 : Us(n-1)
    mpy     w6*w7,A ; G*Us(n-1)

    mov     w3,w7 ;Es(n-1)
    mpy     w6*w7,B ;G*Es(n-1)
    neg     B    ;- G*Es(n-1)
    add     A    ;G*Us(n-1) - G*Es(n-1)
```

接下来呢，就先把 G*Es(n-1)计算出来。这里标注的 Es(n-1)实际上，它就表示 Zs 的低通滤波值。大家要能对应上的话，自己也可以把这个 Es 改为 Zs1。这里把 G*Es(n-1)求出来之后，存放到了累加器 B 中，然后，接着就把 G*Us(n-1) - G*Es(n-1)求出来。所以呢，这里就把累加器 B 中存储的 G*Es(n-1)使用 neg 指令变为了 -G*Es(n-1)。然后呢，再使用一条加法指令，求出 G*Us(n-1) - G*Es(n-1) 。或者大家也可以使用 SUB A 减法指令，不求 neg B 也可以。

```
_EstimateCurrent:
    mov     w1,w6 ;w6 : G
    mov     w2,w7 ;w7 : Us(n-1)
    mpy     w6*w7,A ; G*Us(n-1)

    mov     w3,w7 ;Es(n-1)
    mpy     w6*w7,B ;G*Es(n-1)
    neg     B    ;- G*Es(n-1)
    add     A    ;G*Us(n-1) - G*Es(n-1)

    mov     w4,w7 ;Zs(n-1)
    mpy     w6*w7,B ;G*Zs(n-1)
    neg     B    ;- G*Zs(n-1)
    add     A    ;G*Us(n-1) - G*Es(n-1) - G*Zs(n-1)
```

接着呢，一样的套路，把 G*Zs(n-1)求出来，然后，再把 G*Us(n-1) - G*Es(n-1) - G*Zs(n-1) 求出来，存到累加器 A 中。

```
_EstimateCurrent:
    mov     w1,w6 ;w6 : G
    mov     w2,w7 ;w7 : Us(n-1)
    mpy     w6*w7,A ; G*Us(n-1)

    mov     w3,w7 ;Es(n-1)
    mpy     w6*w7,B ;G*Es(n-1)
    neg     B    ;- G*Es(n-1)
    add     A    ;G*Us(n-1) - G*Es(n-1)

    mov     w4,w7 ;Zs(n-1)
    mpy     w6*w7,B ;G*Zs(n-1)
    neg     B    ;- G*Zs(n-1)
    add     A    ;G*Us(n-1) - G*Es(n-1) - G*Zs(n-1)

    mov     w5,w6 ;F
    mov     w0,w7;Is(n-1)
    mpy     w6*w7,B ;F*Is(n-1)
    add     A    ;F*Is(n-1) + G*Us(n-1) - G*Es(n-1) - G*Zs(n-1)
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

最后，同样的思路，把 F*Is(n-1)求出来，求出来之后，存放到累加器 B 中，再求出 F*Is(n-1) + G*Us(n-1) - G*Es(n-1) - G*Zs(n-1)放到累加器 A 中。然后，我们只需要取出累加器 A 中的数据，存到 w0 寄存器中，就得到返回值，也就是计算得到估算电流了。

```
_EstimateCurrent:
    mov     w1,w6 ;w6 : G
    mov     w2,w7 ;w7 : Us(n-1)
    mpy     w6*w7,A ; G*Us(n-1)

    mov     w3,w7 ;Es(n-1)
    mpy     w6*w7,B ;G*Es(n-1)
    neg     B    ;- G*Es(n-1)
    add     A    ;G*Us(n-1) - G*Es(n-1)

    mov     w4,w7 ;Zs(n-1)
    mpy     w6*w7,B ;G*Zs(n-1)
    neg     B    ;- G*Zs(n-1)
    add     A    ;G*Us(n-1) - G*Es(n-1) - G*Zs(n-1)

    mov     w5,w6 ;F
    mov     w0,w7;Is(n-1)
    mpy     w6*w7,B ;F*Is(n-1)
    add     A    ;F*Is(n-1) + G*Us(n-1) - G*Es(n-1) - G*Zs(n-1)

    sac.r   A,#0,w0
    return
```

所以，整个这段代码的实现思路也很简单，就是相乘，然后累加。

1.电流估算方程的代码实现；
2.电压矢量Us在αβ轴上的投影 Uα，Uβ求取；
3.电流观测器中控制器的代码实现；
4.数字低通滤波器的代码实现(包括自适应滤波系数的求取)；
5.利用估算出来的反电动势，调用数字迭代算法函数，求解估算反电动势合成矢量夹角；
6.角度补偿；
7.估算电转速的求取。

SAC.R 保存舍入后的累加器内容

实现于：	PIC24F	PIC24H	PIC24E	dsPIC30F	dsPIC33F	dsPIC33E
				X	X	X

语法：{ 标号 :} SAC.R Acc, {#Slit4,} Wd
[Wd]
[Wd++]
[Wd--]
[--Wd]
[++Wd]
[Wd + Wb]

操作数：Acc ∈ [A,B]
Slit4 ∈ [-8 ... +7]
Wb ∈ [W0 ... W15]
Wd ∈ [W0 ... W15]

操作：$Shift_{Slit4}$(Acc) （可选）
舍入 (Acc)
(Acc[31:16]) → Wd

受影响的状态位：无

指令编码：

1100	1101	Awww	wrrr	rhhh	dddd

描述：对指定的累加器执行可选的有符号4位移位操作，随后将舍入后的ACCxH（Acc[31:16]）内容存入 Wd。移位位数范围为 -8:7，其中负操作数表明将执行算术左移操作，而正操作数表明将执行算术右移操作。RND 位（CORCON<1>）用于对舍入模式（普通或收敛）进行设定。可使用寄存器直接或间接寻址对 Wd 进行寻址。

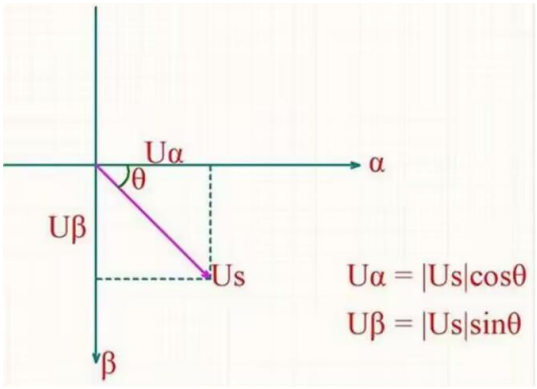
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这句话的意思是将 A[31:16] --> W0 ，A 的高 16 位赋值给 W0。

问：为什么低 16 位数据不要呢？
答：16 位的 q15 精度已经比较高了。

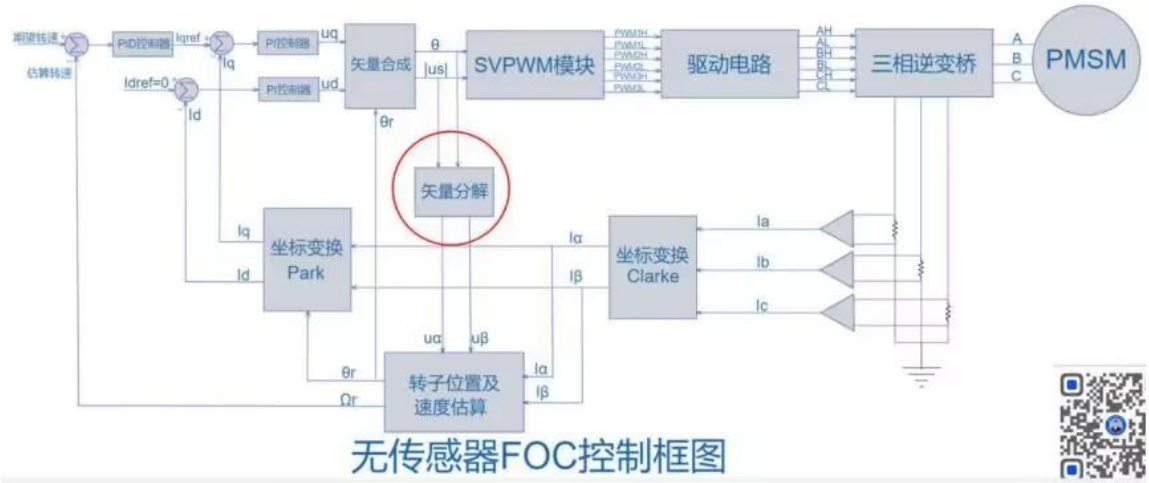
问：汇编中是直接给累加器赋值，其他程序的调用后，再进去这段代码不需要清零吧？
答：累加器是中间过渡变量，并不是输入量，使用完下次还可以用的。

下面我们继续来看第二个部分， 第二个部分，我们要求解 Uα 跟 Uβ 。实际上，矢量如果知道了，我们可以对矢量进行分解。



大家看下这幅图，只需要把 Us 分别往 α 轴， β 轴进行投影，就可以求解出 Uα 跟 Uβ 。

问：角度 θ 还没观测出来，只管测出来反电动势？
答：是的，开始就进行电流观测。Us 的角度是已知的，只要发出 SVPWM,就知道 Us 的角度。开始强拖期间，也是知道 Us 角度的。



这幅框图， 也就是 Us 跟 θ 是清楚的，Us 可以通过 Ud 及 Uq 求解一个平方根得到 θ 的话，可以通过转子位置角 θr + 90° + 超前角 θc 或者 θr - 90° - θc 得到。所以，从框图来看，获取 Uα 跟 Uβ 的话，就是通过矢量分解模块实现的。

问：不是通过观测器去估计角度么？

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

答：估算的是 Es 角度啊，不是 Us 角度。

问：强拖期间 θ 是认为给定对吗，当进入 foc 才采用估算 θ 值对吧？
答：是的，切入闭环后才用。

```
else if(tFoc.Staus == WORK_STATE) //3:Run
{
    IOCON1 = IOCON_HPWM_LPWM;
    IOCON2 = IOCON_HPWM_LPWM;
    IOCON3 = IOCON_HPWM_LPWM;

    GetRotorInfo();
    GetUs();
    if(tFoc.Direction == CCW)
        tCore.qEAngle = tFoc.Eangle_Rotor + EANGLE90 + tUm.qDQAngle;
    else
        tCore.qEAngle = tFoc.Eangle_Rotor - EANGLE90 - tUm.qDQAngle;
    tUm.Percent = tUm.PercentOut;
    UsOut();
}
```

大家再来看下在运行状态下，我们会获取 θ 及 Us 的模，并且呢，我们还要利用 θ 的 sin 跟 cos 值。

```
/*=============================================
void UsOut(void)
{
    CalSinCos(tCore.qEAngle);
    GetUsSector(tCore.qEAngle);
    GetUxyz();
    SvpwmOut();
```

在这个 UsOut 函数中，有查表的操作，所以，相当于 θ 对应的正弦值跟余弦值也是知道的。

```
/*=============================================
void UsOut(void)
{
    CalSinCos(tCore.qEAngle);
    GetUsSector(tCore.qEAngle);
    GetUxyz();
    SvpwmOut();
    GetUalphabeta();
}
/*=====================================版权声
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

那么这样的话，我们把求解 Uα 跟 Uβ 的函数放到查表的下面，就可以得到 Uα 跟 Uβ 了。

```
/*==========================================================*/
/*
    qUalpha = qUs x Cos θ          θ qUs vector angle
    qUalpha = qUs x Sin θ

    Input:  Voltage_Percent     svCos_Value svSin_Value sCos_Value  sSin_Value
    Output: sVs qValpha qVbeta  qVd qVq
*/
void GetUalphabeta(void)
{
    tUm.qUs = (signed int)(tUm.Percent >> 1);

    tUm.qUalphaOld = tUm.qUalpha;
    tUm.qUbetaOld = tUm.qUbeta;

    tUm.qUalpha = MulFrac(tUm.qUs, tCore.qCos);
    tUm.qUbeta = MulFrac(tUm.qUs, tCore.qSin);
}
```

大家看下这个函数，tUm.Percent 这个变量，表示的是电压矢量的模长，它对应着 SVPWM 模块的输入。这个值呢，是无符号 ，我们用 0.16 数据格式(Q16)来表示。但是呢，Uα 跟 Uβ，它是有正有负的，所以呢，需要用 Q15 来表示。因此，这里需要把 Us 的模长转换一下，把 Q16 转换为 Q15。

```
*/
void GetUalphabeta(void)
{
    tUm.qUs = (signed int)(tUm.Percent >> 1);

    tUm.qUalphaOld = tUm.qUalpha;
    tUm.qUbetaOld = tUm.qUbeta;

    tUm.qUalpha = MulFrac(tUm.qUs, tCore.qCos);
    tUm.qUbeta = MulFrac(tUm.qUs, tCore.qSin);
}
/*==========================================版权归张飞实战电子所有
```

这就是红框这句话的目的。

```
void GetUalphabeta(void)
{
    tUm.qUs = (signed int)(tUm.Percent >> 1);

    tUm.qUalphaOld = tUm.qUalpha;
    tUm.qUbetaOld = tUm.qUbeta;

    tUm.qUalpha = MulFrac(tUm.qUs, tCore.qCos);
    tUm.qUbeta = MulFrac(tUm.qUs, tCore.qSin);
}
```

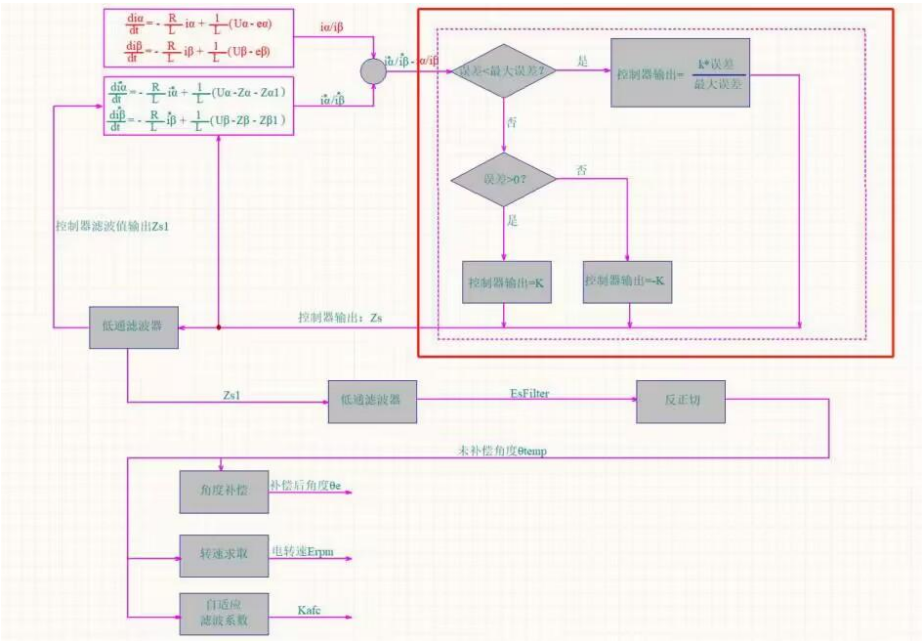
然后呢，后面就是根据公式来求解。上面更新 Old 是为了观测器那边使用。这样的话，Uα 跟 Uβ 的求解代码也实现了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

我们继续往下讲吧

1.电流估算方程的代码实现；
2.电压矢量Us在αβ轴上的投影 Uα，Uβ求取；
3.电流观测器中控制器的代码实现；
4.数字低通滤波器的代码实现(包括自适应滤波系数的求取)；
5.利用估算出来的反电动势，调用数字迭代算法函数，求解估算反电动势合成矢量夹角；
6.角度补偿；
7.估算电转速的求取。

下面呢，我们来看下电流观测器中的控制器代码实现。



也就是这部分　开关　+　线性组合控制的。这个控制器呢，它的输入量是电流的误差，输出量是 Zs。而实际的电流，可以通过电流采样及坐标变换得到。估算电流呢，就通过我们之前讲的那个计算电流估算值的函数来得到。

```
/*==========================================================*/
/*
    Input:  qIalpha qIbeta  qIalpha_Estimate qIbeta_Estimate
            qZErrMax    qZKSlide
    Output: qZalpha     qZbeta
*/
void CalSlideModeZ(void)
{
    signed int qError;
    signed int sTemp;

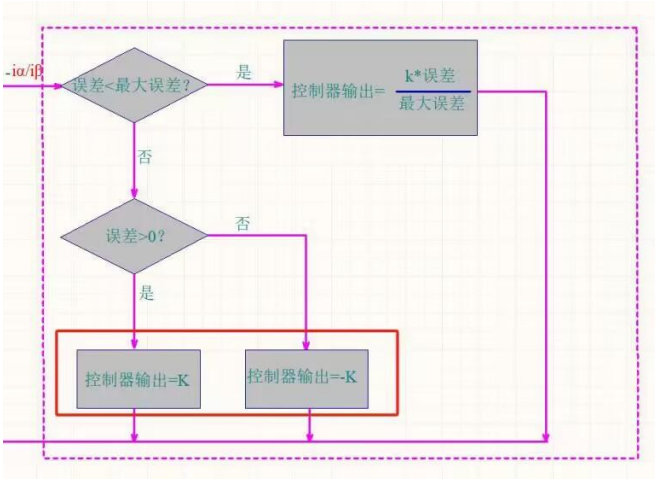
    qError = tObs.qIalphaEstimate - tObs.qIalpha;

    if (qError > tObs.qZErrMax)
    {
        tObs.qZalpha = tObs.qZKSlide;
    }
    else if (qError < -tObs.qZErrMax)
    {
        tObs.qZalpha = -tObs.qZKSlide;
    }
    else
    {
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

我们来看下这个函数，就是开关+线性控制的控制器代码实现，目的是求解出 Zα 跟 Zβ 。

我们先来看下 Zα 的求解。这里呢，分为了三种情况。也就是要判断电流误差是否大于设定的最大误差，或者是否小于负的最大误差，或者是否在线性区内。



大家看，如果判断电流误差是大于正的最大误差，则控制器的输出就取 K。如果判断电流误差小于负的最大误差，则控制器的输出就取-K。

如果判断电流误差是小于正的最大误差，大于负的最大误差的话，就线性计算。

```
void CalSlideModeZ(void)
{
    signed int qError;
    signed int sTemp;

    qError = tObs.qIalphaEstimate - tObs.qIalpha;

    if (qError > tObs.qZErrMax)
    {
        tObs.qZalpha = tObs.qZKSlide;
    }
```

所以，我们第一步是先把电流误差给求出来，然后根据这个误差值，来确定 Zα 。

```
void CalSlideModeZ(void)
{
    signed int qError;
    signed int sTemp;

    qError = tObs.qIalphaEstimate - tObs.qIalpha;

    if (qError > tObs.qZErrMax)
    {
        tObs.qZalpha = tObs.qZKSlide;
    }
    else if (qError < -tObs.qZErrMax)
    {
        tObs.qZalpha = -tObs.qZKSlide;
    }
    else
    {
        sTemp = MulFrac(tObs.qZKSlide,qError);
        tObs.qZalpha = __builtin_divf(sTemp,tObs.qZErrMax);
    }
```

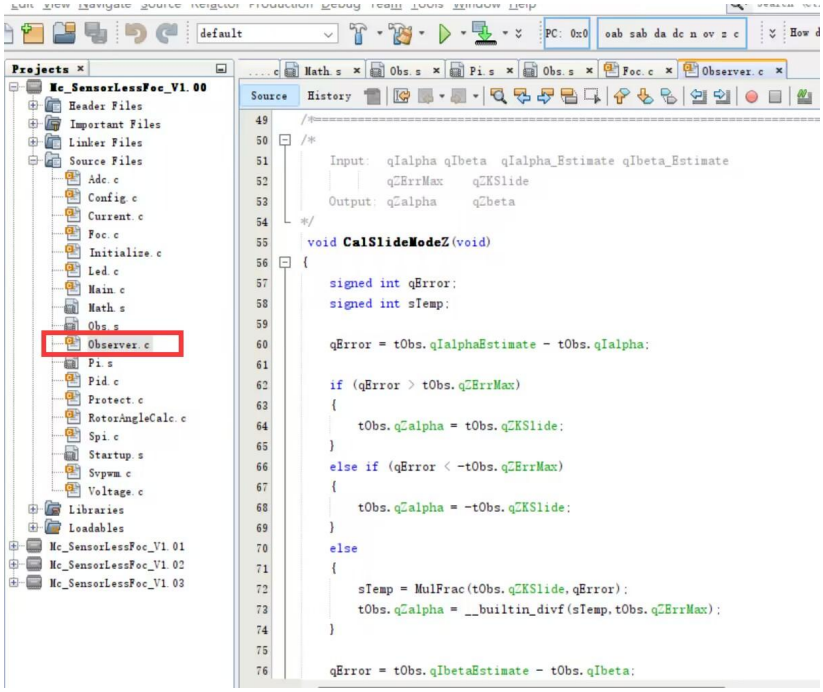
那么，这两种情况的话，是误差大于正的最大误差或者小于负的最小误差的时候，条件成立的话，就直接把+K 或者-K 赋值给 Zα 。

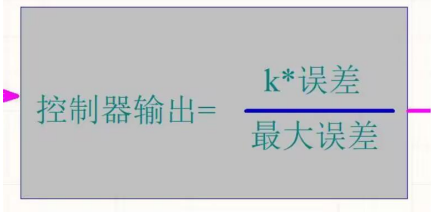
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：设置 K 和最大误差值时，有经验值吗？
答：需要根据估算电流的收敛情况，调整这两个参数，你可以参考课程中的值。

```
else
{
    sTemp = MulFrac(tObs.qZKSlide, qError);
    tObs.qZalpha = __builtin_divf(sTemp, tObs.qZErrMax);
}
```

如果在线性区的话，就执行这段代码。





这呢，会先计算 K*误差，所以，使用了乘法计算的函数。然后呢，需要计算一个除法，这个除法呢，使用的就是内建函数。

问：这个参数调试有一定的规律么？
答：需要观测估算电流和实际电流是否收敛，收敛了，参数就可以了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

__builtin_divf

说明：

计算商 *num*/*den*。如果 *den* 为零，会发生数学错误异常。函数的参数和结果都是无符号的。

原型：

```
unsigned int __builtin_divf(unsigned int num,
unsigned int den);
```

参数：

num 分子
den 分母

返回值：

返回商 *num*/*den* 的无符号整型值。

汇编操作符 / 机器指令：

```
div.f
```

这是内建函数的原型，关于最大误差跟增益 K 的话，我们后面会调试确定的。这里呢，先用变量来表示，先把代码实现。

```
qError = tObs.qIalphaEstimate - tObs.qIalpha;

if (qError > tObs.qZErrMax)
{
    tObs.qZalpha = tObs.qZKSlide;
}
else if (qError < -tObs.qZErrMax)
{
    tObs.qZalpha = -tObs.qZKSlide;
}
else
{
    sTemp = MulFrac(tObs.qZKSlide, qError);
    tObs.qZalpha = __builtin_divf(sTemp, tObs.qZErrMax);
}
```

所以说，这段代码，就是实现 Zα 的求解。

问：这个函数的参数是无符号球，输入参数都是有符号，这段不理解？

答：

DIVF	Wm, Wn	有符号 16/16 位小数除法， Q → W0， R → W1	1	18(2)	228

DIVF 小数除法

实现于：	PIC24F	PIC24H	PIC24E	dsPIC30F	dsPIC33F	dsPIC33E
				X	X	X

语法： {标号 :} DIVF Wm, Wn

操作数： Wm ∈ [W0 ... W15]
Wn ∈ [W2 ... W15]

操作： 0x0 → W0
Wm → W1
W1:W0/Wn → W0
余数 → W1

受影响的状态位： N、OV、Z 和 C

指令编码：	1101	1001	0ttt	t000	0000	ssss

描述： 迭代的有符号 16 位 /16 位小数除法，其中被除数存储在 Wm 中而除数存储在 Wn 中。在操作过程中，W0 将首先被清零而 Wm 将被拷贝至 W1。除法操作的 16 位商将存入 W0，而 16 位余数则被存入 W1。余数的符号将与被除数的符号相同。

必须通过 REPEAT 指令（迭代计数值为 17）执行该指令 18 次以得到正确的商和余数。如果余数为负，则 N 标志将被置 1，否则将被清零。如果除法操作导致溢出，则 OV 标志将被置 1，否则将被清零。如果余数为 0，则 Z 标志将被置 1，否则将被清零。C 标志用来实现除法算法，但不应使用其最终值。

t 位用于选择被除数寄存器。
s 位用于选择除数寄存器。

注 1：为使小数除法有效，Wm 必须小于 Wn。如果 Wm 大于或等于 Wn，将出现意想不到的结果，这是因为小数结果将大于或等于 1.0。当发生这种情况时，OV 状态位将被置 1，而此时的商和余数则不应被使用。
2：以零作为除数时，将在除法执行过程中的第一个指令周期内触发一个运算出错陷阱。
3：该指令在每一个指令周期边界都是可中断的。

指令字数： 1
指令周期数： 18（外加执行 REPEAT 的 1 个指令周期）

汇编指令中说明，这是有符号小数除法。可见 C 语言内建函数的描述是有问题的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

那么 Zβ 的计算，代码的实现方式也是一样的。

```
qError = tObs.qIbetaEstimate - tObs.qIbeta;

if (qError > tObs.qZErrMax)
{
    tObs.qZbeta = tObs.qZKSlide;
}
else if (qError < -tObs.qZErrMax)
{
    tObs.qZbeta = -tObs.qZKSlide;
}
else
{
    sTemp = MulFrac(tObs.qZKSlide, qError);
    tObs.qZbeta = __builtin_divf(sTemp, tObs.qZErrMax);
}
```

这样的话，Zα 跟 Zβ 就求解出来了。

1.电流估算方程的代码实现；
2.电压矢量Us在αβ轴上的投影 Uα，Uβ求取；
3.电流观测器中控制器的代码实现；
4.数字低通滤波器的代码实现(包括自适应滤波系数的求取)；
5.利用估算出来的反电动势，调用数字迭代算法函数，求解估算反电动势合成矢量夹角；
6.角度补偿；
7.估算电转速的求取。

第 3 部分代码实现也就讲完了

1.电流估算方程的代码实现；
2.电压矢量Us在αβ轴上的投影 Uα，Uβ求取；
3.电流观测器中控制器的代码实现；
4.数字低通滤波器的代码实现(包括自适应滤波系数的求取)；
5.利用估算出来的反电动势，调用数字迭代算法函数，求解估算反电动势合成矢量夹角；
6.角度补偿；
7.估算电转速的求取。

第 4 部分，就是我们要对 Zα 跟 Zβ 进行滤波的话，需要使用低通滤波器
第 5 部分，

$$u_o(n) = u_o(n-1) + Ts2\pi fc(u_i(n) - u_o(n-1))$$

滤波器，也就对应这个公式，当然，我们在写代码的时候，可以把这个公式抽象化。把输出写为 y，输入写为 x，滤波系数写为 k。这样的话，这个公式可以变为：
y(n) = y(n-1) + K(x(n) - y(n-1))

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

当然我们的目的，就是要把这个公式用代码实现出来，这个函数呢，这里也是用汇编来写的。

```
/*=====================================================================*
;Input:          W0 W1 W2
;Output:         W0
;y(n) = y(n-1) + K(x(n) - y(n-1))
/*=====================================================================*
_QLowPassFilter:
    mov     w1,w4   ;w4 = x(n)
    mov     w2,w5   ;w5 =  K

    lac     w0,A     ;A = y(n-1)
    mpy     w4*w5,B ;B = K * x(n)
    add     A    ;y(n-1) + K * x(n)

    mov     w0,w4   ; w4 = y(n-1)
    mpy     w4*w5,B ;B = k * y(n-1)
    neg     B    ;B = -k * y(n-1)
    add     A    ;A = y(n-1)+ K * x(n) - k * y(n-1)
    sac.r   A,#0,w0 ;w0 = y(n-1)+ K * x(n) - k * y(n-1)

    return
```

这个函数呢，是写为带三个形参及一个返回值的。

```
extern signed int MulFrac(signed int x,signed int y);
extern signed int CalAtan(signed int y,signed int x);
extern signed int EstimateCurrent(signed int y,signed int g,signed int u,signed int e,signed int z,signed int f);
extern signed int QLowPassFilter(signed int y,signed int x,signed int k);
```

这个呢是 C 文件中调用的声明方式，三个形参，对应的就是 y --> w0 x-->w1 k-->w2 返回值-->w0。

```
/*=====================================================================*/
;Input:          W0 W1 W2
;Output:         W0
;y(n) = y(n-1) + K(x(n) - y(n-1))
/*=====================================================================*/
_QLowPassFilter:
    mov     w1,w4   ;w4 = x(n)
    mov     w2,w5   ;w5 =  K

    lac     w0,A     ;A = y(n-1)
    mpy     w4*w5,B ;B = K * x(n)
    add     A    ;y(n-1) + K * x(n)

    mov     w0,w4   ; w4 = y(n-1)
    mpy     w4*w5,B ;B = k * y(n-1)
    neg     B    ;B = -k * y(n-1)
    add     A    ;A = y(n-1)+ K * x(n) - k * y(n-1)
    sac.r   A,#0,w0 ;w0 = y(n-1)+ K * x(n) - k * y(n-1)

    return
```

这段代码，就对应一阶低通滤波器的具体实现方式。其实，实现的思路跟刚才的估算电流计算是类似的，就是计算乘法跟加法。

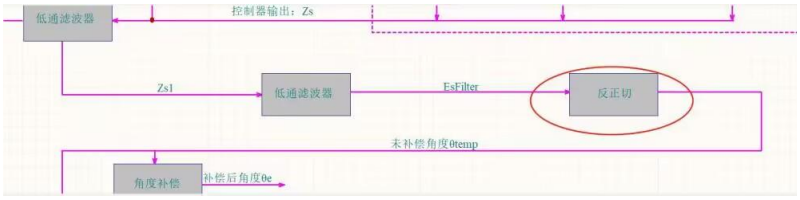
1.电流估算方程的代码实现；
2.电压矢量Us在αβ轴上的投影 Uα，Uβ求取；
3.电流观测器中控制器的代码实现；
4.数字低通滤波器的代码实现(包括自适应滤波系数的求取)；
5.利用估算出来的反电动势，调用数字迭代算法函数，求解估算反电动势合成矢量夹角；
6.角度补偿；
7.估算电转速的求取。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
tObs.qEalpha = QLowPassFilter(tObs.qEalpha, tObs.qZalpha, tObs.qKafc);
tObs.qEbeta = QLowPassFilter(tObs.qEbeta, tObs.qZbeta, tObs.qKafc);

tObs.qEalphaFinal = QLowPassFilter(tObs.qEalphaFinal, tObs.qEalpha, tObs.qKafcFinal);
tObs.qEbetaFinal = QLowPassFilter(tObs.qEbetaFinal, tObs.qEbeta, tObs.qKafcFinal);
```

那么，这里使用了两次低通滤波器。目的呢，是想让估算的反电动势波形更加平滑一些， 这样求解出来的角度变化，也会更平滑一些。当估算的反电动势计算出来之后，我们就会利用这个值去求解估算的角度。



也就是说，这里我们要使用一个反正切，来计算角度。

```
void Observer(void)
{
    PUSHCORCON();
    CORCONbits.SATA = 1;
    CORCONbits.SATB = 1;
    CORCONbits.ACCSAT = 1;      //A B 40bit

    tObs.qUalpha = tUm.qUalphaOld;
    tObs.qUbeta = tUm.qUbetaOld;

    tObs.qIalpha = tIm.qIalpha;
    tObs.qIbeta = tIm.qIbeta;

    tObs.qIalphaEstimate = EstimateCurrent(tObs.qIalphaEstimate, tObs.qG, tObs.qUalpha, tObs.qEalpha, tObs.qZalpha, tObs.qF);
    tObs.qIbetaEstimate = EstimateCurrent(tObs.qIbetaEstimate, tObs.qG, tObs.qUbeta, tObs.qEbeta, tObs.qZbeta, tObs.qF);

    CalSlideModeZ();

    tObs.qEalpha = QLowPassFilter(tObs.qEalpha, tObs.qZalpha, tObs.qKafc);
    tObs.qEbeta = QLowPassFilter(tObs.qEbeta, tObs.qZbeta, tObs.qKafc);

    tObs.qEalphaFinal = QLowPassFilter(tObs.qEalphaFinal, tObs.qEalpha, tObs.qKafcFinal);
    tObs.qEbetaFinal = QLowPassFilter(tObs.qEbetaFinal, tObs.qEbeta, tObs.qKafcFinal);

    CalEangle();

    POPCORCON();
}
```

大家来看下这个观测器函数，按照步骤，先计算估算电流，然后计算控制器的输出，再是对控制器输出滤波。然后，为了得到更加平滑的值，又对控制器的滤波值，再滤了一次波。

```
    tObs.qEalpha = QLowPassFilter(tObs.qEalpha, tObs.qZalpha, tObs.qKafc);
    tObs.qEbeta = QLowPassFilter(tObs.qEbeta, tObs.qZbeta, tObs.qKafc);

    tObs.qEalphaFinal = QLowPassFilter(tObs.qEalphaFinal, tObs.qEalpha, tObs.qKafcFinal);
    tObs.qEbetaFinal = QLowPassFilter(tObs.qEbetaFinal, tObs.qEbeta, tObs.qKafcFinal);

    CalEangle();

    POPCORCON();
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

我们计算没有补偿的角度，就利用这两个值去计算就行。

```
void CalEangle(void)
{
    signed int sTemp;
    signed int qThetaTemp;
    unsigned int Temp;
    UNION_U32 Temp32A;
    UNION_U32 Temp32B;
    UNION_U32 E_Frequency32;

    qThetaTemp = CalAtan(tObs.qEbetaFinal,tObs.qEalphaFinal);

    tObs.qThetaSum += qThetaTemp - tObs.qThetaLast;
    tObs.qThetaLast = qThetaTemp;
```

大家看下，我们之前写的数字迭代算法（反正切）这里又使用了一次，调用之后，就得出没有补偿的估算角度。

```
tObs.qKafc = tObs.qKafcFinal = MulFrac(sTemp,PI_I
tObs.qTheta = qThetaTemp + EANGLE90;
```

而因为我们之前推导过，只要让滤波系数自适应变化，则使用滤波器之后，延迟的相位就是确定的 45°。 我们使用了两次低通滤波，所以这里的相位延迟就是 90°。所以呢，我们补偿的话，就补偿 90°。这个角度的话，也就作为估算的反电动势角度。而转子位置角，跟这个估算反电动势夹角的关系是 $\theta e = \theta r + 90°$。所以，这样的话，也就可以把转子位置角给估算出来了。

今天内容就到这里。谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。